From LLMs to Multi-Agent Systems

A Progressive Guide to Agent Architectures

J.C. Vaught
University of South Carolina
jvaught@sc.edu

February 3, 2026

Abstract

This document provides a comprehensive, progressive guide to multi-agent systems architectures, beginning with foundational concepts of large language models and advancing through increasingly complex architectures. Starting from single-agent systems, we progressively introduce slave hierarchies, orchestration patterns, and multi-layered hierarchies. The guide explores three agent execution models (streaming, batch, and parallel), multiple communication patterns (direct, file-mediated, and hybrid), and diverse control models (imperative, declarative, and hybrid). We examine orchestration cycles ranging from single-pass to iterative and reactive patterns, alongside system topologies including serial, parallel, and tree configurations. Central to this framework are four canonical patterns: pipeline, fan-out/fan-in, master-slave, and feedback loop architectures. Throughout, we provide practical guidance for selecting and combining patterns based on specific task characteristics, computational constraints, and system requirements.

1 What Is an LLM?

Before understanding AI coding agents, we must understand the engine that powers them: the large language model, or LLM. Think of an LLM as a person who can talk but has no hands to manipulate objects, no persistent memory beyond the current conversation, and—usually—no eyes to see the world. This person has read vast amounts of text and learned patterns in language, allowing them to predict what words should come next in any conversation. When you send a message, the LLM generates a response based on everything said so far in the thread, then forgets the interaction the moment it ends.

The lack of memory is worth emphasizing because it contradicts many users' intuitions. When you close your chat with an LLM and open a new conversation tomorrow, the model has no recollection of what you discussed previously. There is no learning, no accumulation of experience, no relationship building. Each conversation is a blank slate. The LLM does not remember your name, your preferences, or the bug you fixed together last week. If the chatbot seems to remember you across sessions, that is the work of external systems maintaining conversation logs and feeding them back as context, not the model itself retaining information. This statelessness has profound implications for agents: without external memory systems, an agent cannot improve its performance on your specific codebase over time unless explicitly given access to logs, documentation, or other persistent records as input.

LLMs vary widely in capability. Some are small and fast but produce simple, sometimes unreliable output—like asking a novice for advice. Others are large and sophisticated, capable of nuanced reasoning and complex problem-solving, though they may take longer to respond. The key insight is that an LLM is fundamentally a prediction engine: given a sequence of text, it predicts the most likely continuation.

This size-capability tradeoff is central to practical deployment. A small model might have a few billion parameters and run efficiently on consumer hardware or respond in milliseconds via cloud APIs. Such models excel at simple tasks like code autocompletion, grammar correction, or answering factual

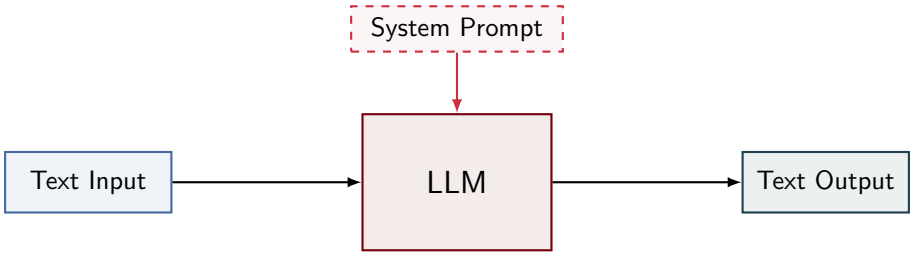

Figure 1: The fundamental LLM interface: text enters, the model processes it under constraints defined by an invisible system prompt, and text emerges. The model retains no information between conversations.

questions. But ask them to debug a subtle concurrency issue or reason about trade-offs in system architecture, and their responses become shallow or incorrect. Conversely, a large model with hundreds of billions of parameters demonstrates impressive reasoning abilities—it can follow multi-step logic, recognize edge cases, and produce creative solutions. However, these capabilities come at a cost: substantial computational resources, higher API fees, and slower response times measured in seconds rather than milliseconds. Choosing the right model for a given task requires balancing accuracy, latency, and cost. In agent systems, where the LLM may be invoked hundreds of times to complete a single user request, these trade-offs compound quickly.

Some modern LLMs have gained "eyes" through multimodal capabilities, meaning they can accept images as input alongside text. This allows them to describe pictures, extract information from diagrams, or analyze visual data. However, the output remains purely textual. The LLM cannot draw, edit pixels, or manipulate files—it can only describe or reason about what it sees.

A critical but invisible component shapes every LLM interaction: the system prompt. Before you type a single word, the LLM has already received instructions from the system designer. This prompt might say "you are a helpful assistant" or "you are a Python expert who writes concise code." The system prompt establishes the LLM's persona, constraints, and priorities. Users never see this text, but it influences every response – often to my chagrin. A playful chatbot and a formal legal assistant may use the same underlying model, differing only in their system prompts.

To illustrate the power of system prompts, consider two interactions with the same LLM. First, with a system prompt reading "You are a creative writing assistant who favors vivid imagery and emotional depth," a user asks, "Describe a rainy day." The model might respond, "The sky wept gray tears onto the cobblestones, each droplet a tiny percussion note in the city's melancholy symphony." Now give the same model a different system prompt: "You are a technical weather analyst. Provide precise, factual descriptions." The identical user query yields, "Precipitation of approximately 5 mm per hour with overcast cloud cover at 800 meters altitude, temperature 12 degrees Celsius, relative humidity 87 percent." Same model, same question, radically different output. The system prompt is not a minor detail—it is the lens through which the model interprets every user request. In agent systems, the system prompt often includes detailed instructions about when to call tools, how to format responses, and what safety checks to perform. A poorly crafted system prompt can render an otherwise capable model nearly useless for agent tasks.

We are simplifying dramatically here but for understanding agents, the essential picture suffices – structured text goes in, the LLM processes it according to learned patterns and its system prompt, and structured text comes out.

2 What Is AI-Assisted Coding?

The simplest application of an LLM to software development is code autocompletion. Here, the LLM is given a system prompt that instructs it to act as a code completion engine: "You are an expert programmer. Given partial code, predict the most likely continuation." The user types code into an editor, and whenever they pause, the LLM receives the text written so far and predicts what should come next. The prediction appears as gray "ghost text" that the user can accept with a keystroke or

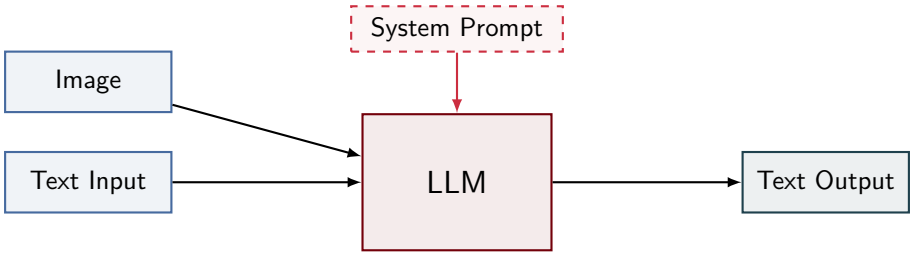

Figure 2: A multimodal LLM accepts images alongside text, but output remains purely textual. The model can describe, analyze, or reason about visual input—it cannot produce images.

ignore by continuing to type.

This is a relatively narrow task. The LLM often has no access to other files in the project, cannot read documentation, and cannot execute or test the code it suggests. It relies entirely on patterns learned during training and the context provided in the current file. If the user has defined a function `calculate_area` earlier in the file, the LLM may predict a call to that function when it seems appropriate. But the LLM is not searching the codebase or verifying correctness—it is simply continuing the established pattern.

Autocomplete represents the minimal intervention of AI in the coding process. The LLM does not take actions, make decisions, or interact with the development environment. It offers a single prediction, and the human decides whether to accept it. This simplicity is both a strength and a limitation. Autocomplete is fast, unobtrusive, and requires no additional infrastructure. But it cannot refactor code across multiple files, fetch API documentation, or debug a runtime error. For those tasks, we need something more capable: an agent.

3 What Is an AI Coding Agent?

An agent transforms an LLM from a passive predictor into an active collaborator. The LLM itself does not change—it still receives text and produces text. What changes is the infrastructure wrapped around it. A coding agent consists of the LLM plus a wrapper program that orchestrates interactions between the model, the development environment, and the user.

The wrapper provides the LLM with two critical additions beyond the system prompt: tool definitions and an execution environment. Tool definitions describe actions the agent can take, such as "read a file," "run a command," or "search the codebase." Each definition specifies the tool's name, parameters, and purpose. The LLM does not directly execute these tools—it cannot. Instead, when the LLM determines that a tool would help answer the user's request, it outputs a structured tool call in its response. The wrapper program parses this output, detects the tool call, executes the requested action in the real environment, and feeds the result back to the LLM as new context.

This creates a loop. The user sends a request. The LLM considers the request alongside its system prompt, available tools, and conversation history, then produces a response. If the response contains a tool call, the wrapper executes it, appends the result to the conversation, and invokes the LLM again. The LLM now has additional information and can either call another tool or provide a final answer. This cycle continues until the LLM produces a response with no tool calls, signaling that the task is complete.

The wrapper exerts control over what the agent can and cannot do. It decides which tools are available, enforces safety constraints (preventing destructive operations without confirmation), and manages the user interface. If the LLM requests a tool that does not exist or provides invalid parameters, the wrapper catches the error and may return a message to the LLM explaining the problem. The wrapper also handles concurrency, timeouts, and resource limits—concerns far beyond the LLM's domain.

From the user's perspective, the agent appears to act autonomously. They might ask, "Fix the bug in `parser.py`," and observe the agent reading the file, analyzing the code, running tests, editing the file, and re-running tests to confirm the fix. But this autonomy is an illusion choreographed by the

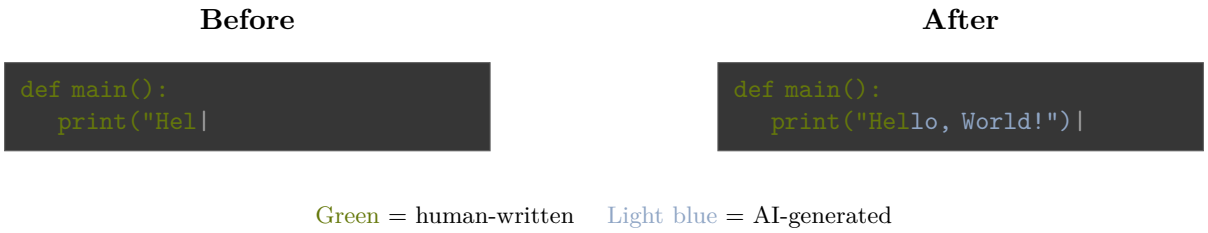

Figure 3: AI-assisted code completion: the user types partial code, the LLM predicts the continuation, and the suggestion appears inline. The model operates purely on visible context with no access to files, tools, or execution environments.

wrapper. The LLM is still only predicting text, albeit text that encodes tool calls. The wrapper translates those predictions into real-world actions.

This architecture is extrmeely powerful. By giving the LLM access to file systems, compilers, debuggers, version control, and external APIs through tool calls, the agent can perform tasks that would be impossible for raw autocompletion. Yet it inherits all the LLM's limitations: no persistent memory across sessions, no ability to learn from mistakes without retraining, and occasional lapses in reasoning. The agent is not intelligent in a human sense—it is an LLM augmented with the ability to manipulate its environment, guided by a carefully crafted system prompt and constrained by the tools its wrapper provides.

4 Agent Workflow — Streaming Execution

In a streaming execution model, the wrapper monitors the LLM's output token-by-token as it is generated. As soon as a complete tool call is detected in the stream, the wrapper immediately pauses the LLM's generation, executes the tool, appends the result to the conversation context, and resumes generation. This cycle repeats until the LLM emits a terminate signal indicating no further actions are required.

The primary advantage of streaming execution is responsiveness. Users see output appearing in real time rather than waiting for a complete response. Tools fire as soon as they are needed, minimizing latency between decision and action. This creates a more interactive experience where intermediate results can inform subsequent reasoning steps without waiting for the entire generation to complete.

However, streaming execution requires more sophisticated infrastructure. The wrapper must parse incomplete output, maintain state across pause-resume cycles, and handle potential errors in partially generated tool calls. The LLM must also be capable of resuming generation coherently after external results are injected mid-stream.

Figure 5 illustrates this workflow. The LLM generates output continuously, monitored for tool calls. Upon detection, execution is handed off to the tool, results are appended, and generation resumes. The process terminates only when the LLM explicitly signals completion.

5 Agent Workflow — Batch Execution

Batch execution follows a simpler but less responsive pattern. The LLM generates its entire response in one pass until it naturally stops. Only then does the wrapper parse the complete output, extract all tool calls present, and execute them serially one after another. All tool results are collected and fed back to the LLM simultaneously, prompting a new generation cycle. This process repeats until the LLM produces output with no tool calls, indicating task completion.

This is the classic chatbot model familiar from interactive coding assistants: the agent writes code, attempts to run it, receives error messages, and tries again. Batch execution is significantly simpler to implement than streaming because it requires no real-time parsing or state management. The wrapper operates on complete, well-formed outputs and can use standard parsing techniques.

The trade-off is latency. Users must wait for the entire generation to complete before any tools execute, and all tools must finish before the next

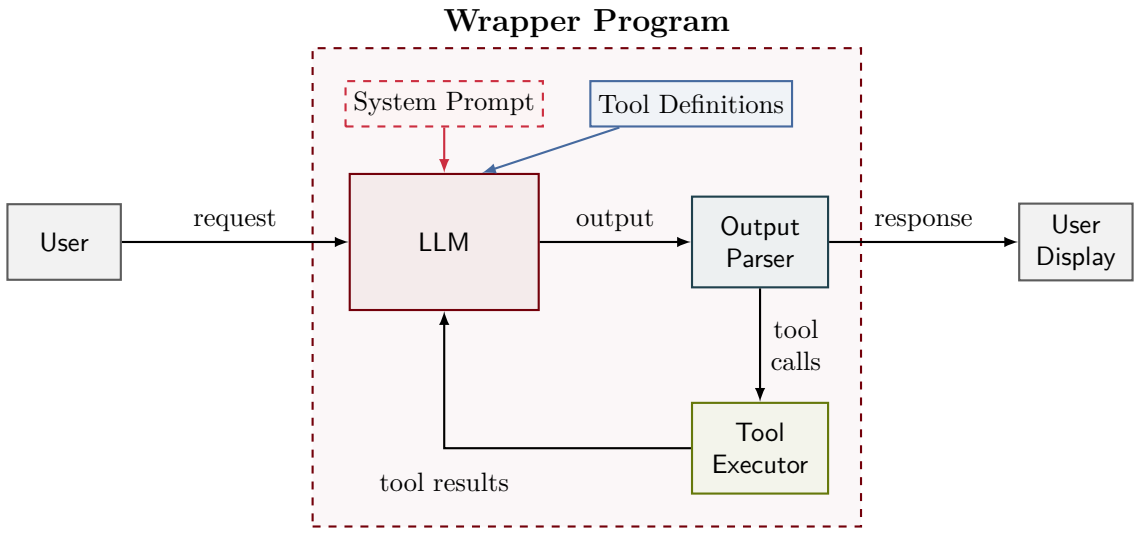

Figure 4: Architecture of an AI coding agent. The wrapper program mediates all interactions between the user and the LLM, parsing outputs to detect tool calls, executing those calls in the development environment, and feeding results back to the model. The LLM operates under constraints defined by its system prompt and the tools made available by the wrapper.

reasoning step begins. For complex tasks requiring multiple tool invocations, this can result in noticeable delays between observable progress. However, for tasks where tool calls are infrequent or where thoughtful planning precedes action, batch execution provides a robust and maintainable architecture.

Figure 6 shows the batch workflow. The LLM produces complete output, which is parsed to extract all tool calls. These are executed serially, producing a combined result set that is fed back to the LLM to trigger the next generation cycle.

6 Agent Workflow — Parallel Execution

Parallel execution extends the batch model by introducing concurrency. Each tool call generated by the LLM can carry a flag indicating whether it can execute in parallel with other operations or requires blocking synchronous execution. When the LLM generates a tool call marked for parallel execution, the wrapper launches it asynchronously and allows the LLM to continue generating additional output or tool calls. When a non-parallel tool call is encountered or the LLM emits a terminate signal, the wrapper stops, waits for all pending parallel operations to complete, executes any blocking tool, and feeds all accumulated results back to the LLM.

This architecture combines the responsiveness of streaming execution with the throughput advantages of concurrency. Multiple independent operations—such as reading several files, running tests, or querying external APIs—can proceed simultaneously without blocking each other or the LLM's reasoning process. Critical operations that depend on prior results or require exclusive access to resources can still be executed synchronously to maintain correctness.

The complexity lies in managing parallel task lifetimes, handling errors from concurrent operations, and ensuring deterministic behavior when multiple results arrive out of order. The wrapper must track which parallel operations are outstanding, aggregate their results, and present them coherently to the LLM. The LLM itself must understand which operations can safely run concurrently and which require sequential ordering.

Figure 7 illustrates this model. Tools marked with a parallel flag execute concurrently while generation continues. A synchronization barrier is inserted before any blocking tool, ensuring all prior parallel operations complete before the blocking operation begins. All results are then returned to the LLM together, maintaining a consistent view of the execution state.

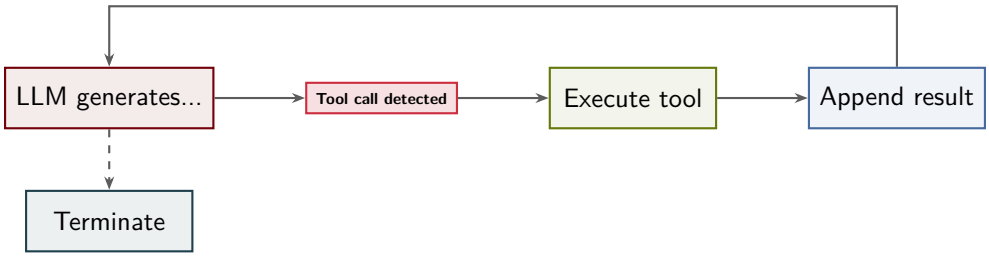

Figure 5: Streaming execution workflow. The LLM generates output incrementally; as soon as a tool call is detected, execution pauses, the tool runs, results are appended, and generation resumes. The process repeats until the LLM emits a terminate signal.

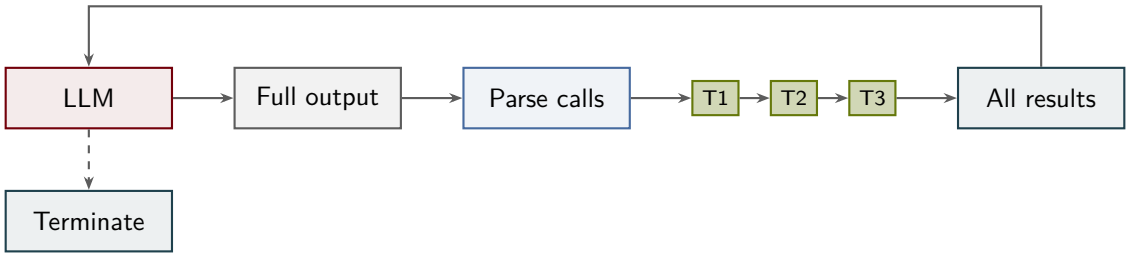

Figure 6: Batch execution workflow. The LLM generates a complete response, tool calls are parsed and executed serially (T1, T2, T3), and all results are fed back to trigger the next reasoning cycle.

7 What Is a Slave?

A slave is architecturally identical to the agent described in Section 3. It possesses the same core components: an LLM capable of reasoning and tool use, a system prompt that defines its behavior, and access to tools for interacting with the environment. The critical distinction is not in its internal structure but in its invocation: a slave is called by another agent rather than directly by a human user.

This architectural equivalence allows for significant flexibility in deployment. A slave may run the same model as its master, but more commonly it uses a different model optimized for its specialized task. A master agent handling general orchestration might use a large, capable model, while delegating narrowly-scoped work to slaves running smaller, faster, or domain-tuned models. Similarly, the system prompt of a slave is typically specialized for a particular domain—code refactoring, test execution, documentation generation, data validation—rather than the general-purpose instructions given to a primary agent.

Tool access is also scoped to the slave's responsibilities. Where a master agent might have access to the full suite of development tools, a slave focused on testing might be restricted to test execution frameworks and assertion libraries. This scoping reduces complexity, improves safety, and allows the slave's reasoning to focus on a well-defined problem space.

8 Slave Workflow

The lifecycle of a slave begins when the master agent constructs a detailed prompt describing the task to be performed. This prompt is delivered to the slave, where it is combined with the slave's own system prompt to form the complete context for execution. The system prompt provides domain expertise and behavioral constraints, while the master's prompt supplies the specific task instance and any relevant data.

Once initialized, the slave executes its task using any of the three reasoning patterns described in Sections 4–6. A slave for code refactoring might use streaming mode to interactively rewrite functions, a testing slave might operate in batch mode to execute a suite and compile results, and a documentation slave might parallelize the generation of API reference sections. The choice of execution pattern is determined by the slave's design and the nature of its task, not by the fact that it is a slave.

Upon completion, the slave returns control to its

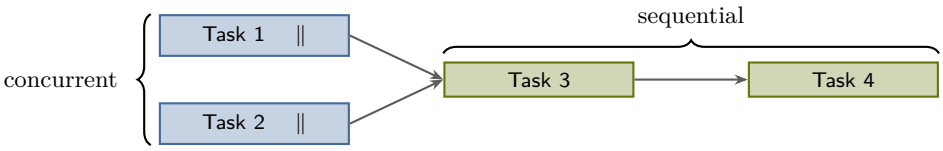

Figure 7: Parallel execution: tasks marked || run concurrently. Once both complete, subsequent tasks execute sequentially. The agent decides which tasks are independent and can safely overlap.

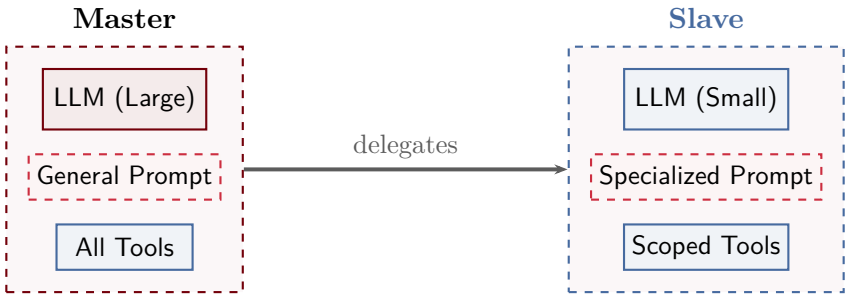

Figure 8: A slave shares the same architecture as a standard agent (Section 3) but differs in its model selection, system prompt specialization, and tool access scope.

master. The exact form of this return varies: the slave may include its result directly in a completion message, write outputs to disk and signal completion, or employ a hybrid approach. Regardless of the mechanism, the master receives notification that the slave has finished and can then integrate the result into its own reasoning process. This integration might involve validating the slave's output, using it as input to another slave, or incorporating it into a final deliverable for the user.

9 How Do We Manage Slaves?

The master-slave pattern provides the foundation for managing multiple slaves. In this architecture, a master agent serves as the coordinator, responsible for decomposing complex work into discrete tasks and distributing those tasks to specialized slaves. The master does not perform the work itself; instead, it coordinates, tracks progress, and synthesizes results.

A critical element of this pattern is the completion signal. Regardless of the communication mechanism employed—direct return, file-mediated output, or hybrid approaches—every slave must signal when it has finished its assigned task. This signal may be as simple as a boolean flag or as rich as a structured status message, but its presence is non-negotiable. Without reliable completion signals, the master cannot accurately track system state or make informed decisions about subsequent actions.

The master maintains state about the entire pool of slaves. This state includes which slaves are currently executing, which have completed their tasks, what results have been received, and what work remains to be distributed. When a slave signals completion, the master updates its internal model, evaluates whether dependencies have been satisfied, and determines the next steps. This might involve launching additional slaves to handle newly unblocked tasks, aggregating results from completed slaves to produce a final output, or escalating issues when a slave reports failure.

10 Communication Patterns

The communication pattern between master and slave determines how results flow back to the master. Three patterns dominate in practice, each suited to different types of tasks and output characteristics.

Direct return embeds the result directly in the completion message. This pattern is appropriate when the master requires the slave's answer to make an immediate decision. A testing slave might return a simple pass/fail verdict, an installation slave might report success or error codes, and a query slave might return a small dataset or configuration

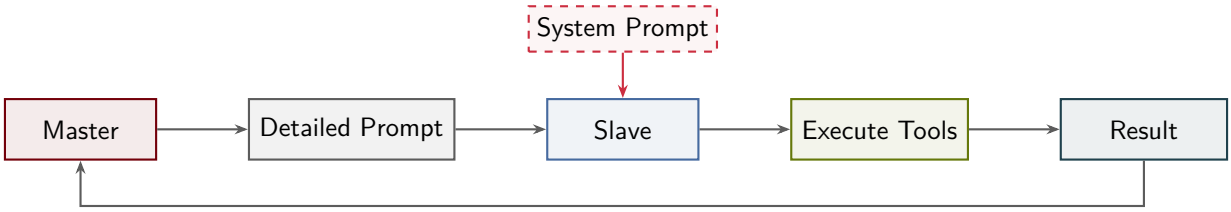

Figure 9: Slave lifecycle. The master crafts a detailed prompt, the slave combines it with its own system prompt, executes tools using any execution pattern, and returns results to the master.

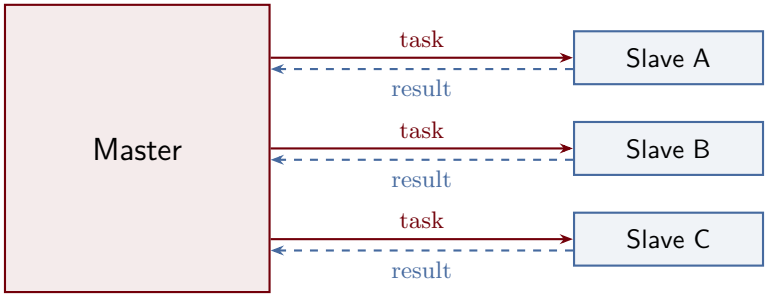

Figure 10: The master-slave pattern. The master assigns tasks to slaves via straight delegation (solid) and receives results back (dashed). Each slave operates independently with its own context.

value. The advantage is immediacy and simplicity: the master receives everything it needs in a single message. The disadvantage is scalability: large results bloat messages and may exceed protocol limits.

File-mediated communication separates the completion signal from the result. The slave writes its output to disk—a dataset, a rendered image, a binary artifact, newly generated source files—and then sends a minimal completion message to the master. The master can then read the file if and when it needs the data. This pattern is essential for large outputs that would be impractical to transmit inline. It also naturally handles side effects: when a code generation slave produces new source files, those files are the deliverable, not a message describing them. The slave still sends a completion flag to notify the master that the work is done, but the substantive output lives on disk.

Hybrid communication, the most common pattern in production systems, combines both approaches. The slave writes detailed artifacts to disk and simultaneously returns a summary in its completion message. A test runner might write full logs and stack traces to a file while returning "14 passed, 2 failed" to the master. A data processing slave might serialize a large DataFrame to disk while returning basic statistics (row count, schema, validation errors) inline. This gives the master enough information to make decisions without forcing it to parse large files, while preserving complete outputs for debugging, auditing, or downstream consumption.

11 Master Control Models

The master's control model determines the distribution of intelligence between master and slave. Three models exist along a spectrum from fully centralized to fully distributed decision-making.

In the **imperative model**, the master agent dictates exact steps for the slave to execute. The master's prompt might read: "Profile the function `compute_similarity`, identify the bottleneck in the inner loop, replace the nested iteration with a vectorized NumPy operation, and verify the output matches the original implementation." Intelligence resides in the master, which has performed the analysis and determined the precise remedy. The slave is a capable executor but makes no strategic decisions. This model offers high predictability and tight control, making it suitable for tasks where the master has domain expertise and the solution path is well understood. The cost is brittleness: if the prescribed steps encounter unexpected conditions, the slave has no autonomy to adapt.

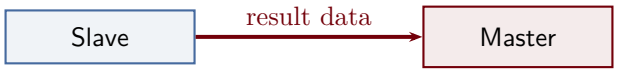

Figure 11: Direct return: the slave embeds its result in the completion message. Simple and immediate, suited for small results like pass/fail verdicts or status codes.

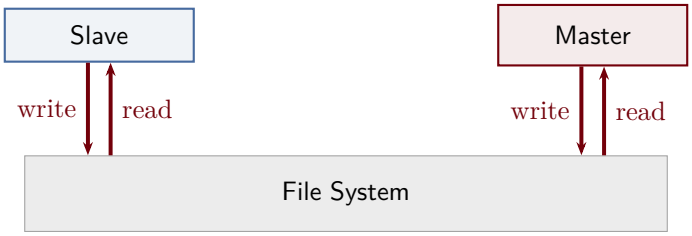

Figure 12: File-mediated: the slave writes output to a shared file system and sends only a completion signal. The master reads the file when needed. Suited for large artifacts, binaries, or generated source files.

The **declarative model** inverts this relationship. The master states only the goal: "Make the function `compute_similarity` more efficient." The slave receives this objective and autonomously determines how to achieve it. It might profile the code, analyze algorithmic complexity, explore alternative data structures, or apply compiler optimizations—all without further direction from the master. Intelligence lives in the slave, which must possess both domain expertise and strategic reasoning. This model offers maximum flexibility and can discover solutions the master never considered. The cost is reduced predictability and control: the master cannot easily constrain the solution space or guarantee particular approaches.

The **hybrid model**, dominant in practice, blends goal specification with constraints. The master might instruct: "Optimize the function `compute_similarity`. The optimization must pass all existing tests without modification, must not introduce new dependencies, and should target a 2× speedup." This shares intelligence between master and slave. The master encodes domain knowledge and project constraints, while the slave applies technical expertise to find a solution within those bounds. The balance is tunable: more constraints shift control toward the imperative end, fewer constraints toward the declarative end.

Table 1 summarizes the tradeoffs. Imperative control offers predictability at the cost of flexibility. Declarative control enables creative problem-solving but sacrifices determinism. Hybrid control, by allowing the master to tune the constraint set, provides a practical middle ground for most production agent systems.

12 The Master Cycle

Every master, regardless of its sophistication, executes some variant of a fundamental five-step loop: **Plan**, **Delegate**, **Wait**, **Collect**, and **Decide**. This cycle forms the operational heartbeat of master-slave architectures. The master begins by planning what work needs to be done, delegates tasks to subordinate agents, waits for those agents to complete their assignments, collects the results, and then decides what to do next — synthesize a final answer, iterate with refined tasks, or terminate.

The differences between orchestration strategies lie not in whether these steps occur, but in *how* each step executes and whether the cycle repeats. A simple master may traverse the cycle exactly once, delegating all tasks upfront and terminating after collecting results. A sophisticated master may iterate through multiple cycles, refining its plan based on what previous slaves discovered. A reactive master may skip explicit planning altogether, instead delegating minimal initial work and deciding on-the-fly how to respond to whatever comes back.

It is important to recognize that this five-step cycle is an **abstraction**, not a rigid implementation requirement. Real masters may combine steps, reorder operations, or execute multiple steps con-

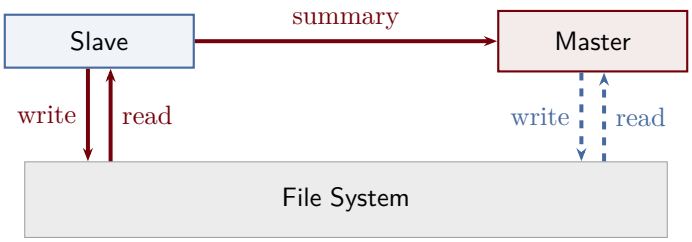

Figure 13: Hybrid (most common): the slave writes detailed artifacts to a shared file system and returns a concise summary directly to the master. Example: a test runner writes full logs to a file but returns "14 passed, 2 failed."

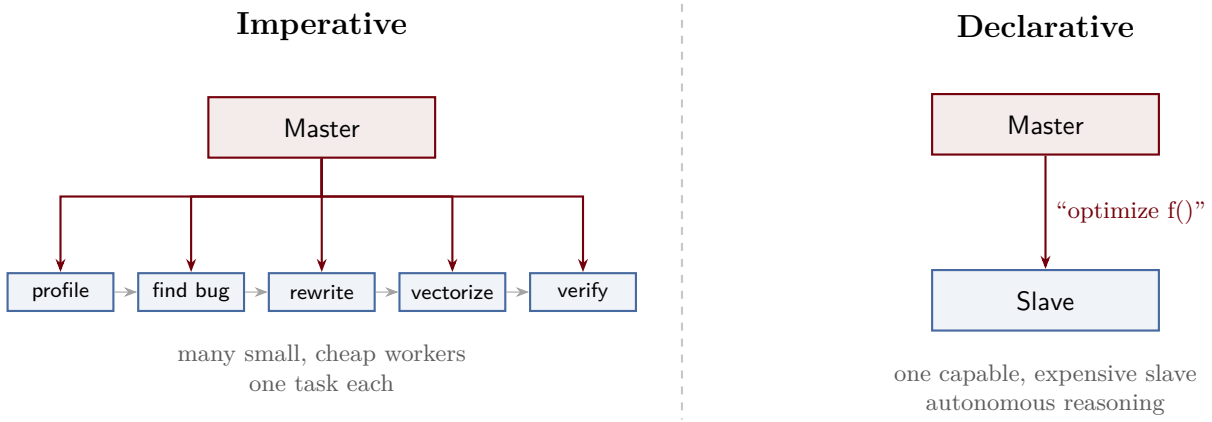

Figure 14: Imperative vs. declarative control models. With imperative control, the master decomposes work into micro-tasks and delegates each to a small, inexpensive slave. With declarative control, the master states the goal and a single capable slave reasons autonomously. The master's intelligence remains constant; what changes is the granularity of delegation.

currently. For example, a master might overlap the Wait and Delegate phases by launching new slaves while waiting for earlier slaves to complete, effectively pipelining the cycle. A reactive master might merge Plan and Decide into a single decision function that simultaneously evaluates results and determines next actions. Some implementations may delegate tasks to slaves that themselves run mini-orchestration cycles, creating recursive delegation patterns that do not cleanly map to the linear five-step model. Despite these variations, the conceptual model remains useful: every master must identify work to be done, assign that work to agents, observe the outcomes, and determine whether to continue or terminate. The five-step abstraction provides a shared vocabulary for reasoning about orchestration strategies, even when the underlying implementations diverge from the idealized cycle.

Figure 15 illustrates the single-pass cycle: a straight line from Plan to Done with no feedback. Figure 16 shows the iterative variant, where a feedback loop from Decide back to Plan becomes the central mechanism for quality improvement and error recovery — the master explicitly routes control back to the planning phase for another round of delegation.

The sections that follow explore three fundamental orchestration patterns: single-pass orchestration (Section 13), which traverses the cycle once and terminates; iterative orchestration (Section 14), which repeats the cycle until a quality threshold or iteration limit is reached; and reactive orchestration (Section 15), which abandons upfront planning in favor of event-driven adaptation. Each pattern represents a different trade-off between simplicity, robustness, and latency.

Table 1: Comparison of master control models.

Property	Imperative	Declarative	Hybrid
Intelligence location	Master	Slave	Shared
Message size	Large	Small	Medium
Worker autonomy	Low	High	Medium
Predictability	High	Low	Medium
Flexibility	Low	High	Medium
Best use case	Well-understood tasks with known solutions	Open-ended problems requiring creativity	Constrained optimization with clear requirements

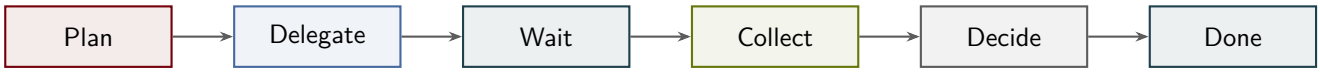

Figure 15: Single-pass orchestration cycle. The master traverses the five-step sequence Plan → Delegate → Wait → Collect → Decide exactly once, then terminates. There is no feedback loop and no opportunity to refine the plan based on slave results.

13 Single-Pass Orchestration

The simplest orchestration model is the single-pass architecture: one traversal through the cycle, no iteration, no recovery. The master plans once, delegates all identified tasks to slaves, waits for completion, collects results, synthesizes a final answer in the Decide phase, and terminates. This design prioritizes latency and simplicity. There are no feedback loops, no quality checks, and no opportunity to refine the plan based on what slaves discover.

Single-pass orchestration works well when the problem is well-understood, the initial decomposition is correct, and slaves reliably succeed. But these conditions rarely hold in practice. If the master misjudges the task breakdown — perhaps underestimating complexity or failing to anticipate dependencies — it has no recourse. If a slave fails due to an unavailable tool, an API timeout, or an unexpected data format, the master cannot retry, cannot delegate diagnostic work, and cannot adjust the plan. The master simply moves forward with whatever results it has.

This leads to a critical failure mode: **the master either gives up entirely or synthesizes a partial or hallucinated answer**. When faced with incomplete results, a single-pass master has two options. The first is to admit defeat, returning a message like "I was unable to complete your task." The second, more insidious option is to produce an answer anyway, filling gaps with plausible-sounding but incorrect information synthesized from the fragments it managed to collect. Both outcomes are unacceptable for production systems, yet single-pass architectures are common in early-generation coding agents and task assistants.

The prevalence of single-pass orchestration in current coding agents explains many user frustrations with AI-assisted development tools. Consider a concrete example: a user asks an agent to refactor a Python module to use asynchronous I/O. The master decomposes this into three tasks: identify all synchronous I/O calls, update function signatures to use async/await, and update all call sites to await the new async functions. It delegates these tasks to three slaves in parallel. The slaves complete their assignments and return transformed code. The master collects the results, synthesizes them into a final patch, and presents it to the user. The user applies the patch, runs their test suite, and discovers that the application crashes on startup. The root cause: the refactoring also required updating configuration files that specify event loop policies, and the master never identified this as a required task. Because the master never considered configuration files in its initial decomposition, no slave was delegated to handle them, and the gap remained invisible until runtime. A single-pass master has no mechanism to detect such gaps — it cannot observe the test failure, cannot spawn a diagnostic slave to inves-

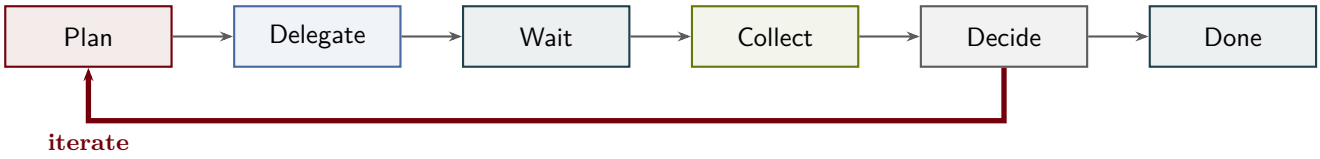

Figure 16: Iterative orchestration cycle. The same five-step sequence, but with a feedback loop (bold arrow) from Decide back to Plan. After collecting results, the master evaluates quality and either terminates or iterates with refined tasks, enabling error recovery and progressive refinement.

tigate, and cannot iterate to fill the missing piece. The result is broken code and a frustrated user who must either manually debug the issue or restart the entire interaction with more explicit instructions.

This failure mode is not an edge case; it is **the fundamental limitation of single-pass orchestration**. When the initial task decomposition is incomplete or incorrect, there is no recovery path. The master commits fully to its upfront plan, and any flaws in that plan propagate directly into the final output. Users experience this as the agent "not understanding the full scope of the task," "missing obvious dependencies," or "breaking working code." From the master's perspective, however, it executed flawlessly: it planned, delegated, collected, and synthesized exactly as designed. The architecture itself provides no feedback loop through which the master might learn that its plan was insufficient.

The appeal of single-pass orchestration is its simplicity. There is no complex control flow, no iteration counters, no quality evaluation logic. The master is a straightforward pipeline: decompose, dispatch, wait, synthesize, done. But this simplicity comes at the cost of brittleness. Single-pass masters fail silently, abandon tasks at the first obstacle, and provide no mechanism for the user or the system to understand why a failure occurred or how it might be remedied.

Figure 15 illustrates this flow. Once the master reaches the Decide phase, it either synthesizes a final answer and terminates successfully, or — because there is no path back to planning, no retry logic, and no recovery mechanism — it gives up entirely or produces a partial, potentially hallucinated result.

14 Iterative Orchestration

Iterative orchestration extends the base cycle by adding explicit feedback: after collecting results, the Decide phase evaluates whether the work is complete and of sufficient quality. If not, the master plans *new* tasks informed by what it learned from previous slaves, delegates those tasks, and repeats the cycle. This continues until one of three termination conditions is met: a **quality threshold** is reached, a **maximum iteration count** is exceeded, or the master determines that **no new tasks are necessary**.

The quality threshold termination condition requires the master to measure output quality against some objective or heuristic standard. In a coding agent, this might mean running a test suite and terminating when all tests pass, or invoking a static analyzer and iterating until all warnings are resolved. In a research assistant, it might mean evaluating a generated summary against a rubric of completeness and accuracy, using either a separate verification agent or a scoring function. The challenge is that many tasks lack clear quality metrics. How does a master measure whether a code refactoring is "good enough"? One approach is to define proxy metrics: compilation succeeds, tests pass, code coverage remains stable, and static analysis reports no new issues. Another approach is to delegate the quality decision itself to a specialized verification agent that reviews the output and returns a binary approve/reject signal. Either way, the master must implement domain-specific evaluation logic in its Decide phase.

The maximum iteration count termination condition is a safety mechanism to prevent infinite loops. If the master cannot satisfy its quality threshold — perhaps the task is genuinely impossible, or the slaves are repeatedly failing for systemic reasons — it must eventually give up rather than burning resources indefinitely. The iteration limit is typically a configuration parameter, tuned based on task complexity and acceptable latency. A simple task might cap at three iterations; a complex multi-step workflow might allow ten or twenty. When the

limit is reached, the master either returns the best result achieved so far (a degraded success) or admits failure, depending on whether partial progress is useful to the user.

The no-new-tasks termination condition represents successful completion through exhaustion: the master has identified and executed all necessary work. This is distinct from reaching a quality threshold, because the master may complete all planned tasks and still fail to meet quality goals, triggering further iteration. Conversely, the master might reach the quality threshold before exhausting its task list, if earlier tasks turned out to be unnecessary or if slaves exceeded expectations. The Decide phase must evaluate both dimensions — quality of output and completeness of task list — to determine whether to iterate or terminate.

Compared to single-pass orchestration, iterative orchestration is **strictly more capable** but incurs higher token and latency costs. An iterative master can recover from initial planning errors, adapt to unexpected slave results, and refine its approach through successive cycles. However, each cycle requires at least one additional LLM invocation in the Plan phase, plus the token costs of delegating new slaves and evaluating their results. Latency multiplies linearly with iteration count, as each cycle must complete before the next begins. For tasks where a correct upfront plan is achievable, single-pass orchestration is more efficient. For tasks where the problem structure is uncertain or slaves may fail unpredictably, the added cost of iteration is justified by the increased likelihood of successful completion.

The iterative model trades latency for quality and resilience. Where a single-pass master fails silently or gives up at the first obstacle, an iterative master can retry failed tasks with refined prompts, delegate diagnostic work to understand why a failure occurred, or spawn additional slaves to fill gaps discovered during result synthesis. The master accumulates knowledge across iterations: each cycle refines the plan based on what previous slaves discovered, what errors they encountered, and what intermediate results they produced.

Consider a concrete example: a master tasked with implementing a feature in a large codebase. In iteration one, it delegates a slave to locate the relevant module and another to identify dependencies. The first slave succeeds, but the second reports that the dependency graph is more complex than expected. In iteration two, the master spawns additional slaves to trace transitive dependencies and verify API contracts. One of these slaves discovers a breaking change in an upstream library. In iteration three, the master delegates a slave to implement a compatibility shim. Only after this third cycle does the Decide phase determine that all preconditions are satisfied and approve the final implementation.

This example illustrates the core advantage of iterative orchestration: **it transforms brittle failure into adaptive problem-solving**. The master does not need a perfect plan upfront; it can discover the true structure of the problem through successive refinement. Failed slaves provide information — error messages, partial results, diagnostic traces — that the master uses to improve its next attempt.

However, iteration introduces new challenges. The master must maintain state across cycles, tracking which tasks have been attempted, which have succeeded, and which need refinement. It must implement a quality evaluation function in the Decide phase, which may require domain-specific heuristics or external verification tools. And it must guard against infinite loops, implementing termination logic that balances thoroughness with resource constraints.

Figure 16 shows the iterative cycle with the feedback loop from Decide to Plan rendered as a bold arrow labeled “iterate.” This loop is the defining characteristic of iterative orchestration. The master also includes an exit path labeled “done,” taken when a termination condition is met. The challenge in implementing iterative orchestration lies in deciding when to iterate and when to exit — a decision that depends on the task domain, the quality of intermediate results, and the cost of additional cycles.

15 Reactive Orchestration

Reactive orchestration abandons upfront planning altogether. Instead of decomposing the entire task before delegating any work, a reactive master starts with a minimal initial action — perhaps a single diagnostic query or a broad exploratory task — and then reacts to whatever comes back. If a slave succeeds, the master decides whether to continue along

the same path or terminate. If a slave returns an error, the master spawns a diagnostic slave to investigate. If a slave discovers something unexpected — a missing file, an API incompatibility, a hidden dependency — the master adjusts its approach, delegating new tasks to handle the surprise.

This is **event-driven** orchestration, not plan-driven. The master maintains minimal state and makes local decisions based on the most recent slave output. It does not attempt to construct a complete task graph upfront; instead, it builds the graph incrementally, one node at a time, guided by the results it observes. This makes reactive orchestration particularly well-suited for exploratory tasks where the problem space is poorly understood: debugging, codebase exploration, open-ended research, or adversarial testing.

Consider debugging a test failure in an unfamiliar codebase. A plan-driven master would attempt to decompose the problem upfront: read the test file, identify the failing assertion, trace dependencies, inspect relevant modules, and so on. But this decomposition requires knowledge the master does not yet have — which modules are relevant? What dependencies matter? A reactive master, by contrast, begins with a single action: run the test and capture the failure output. It then reacts to that output. If the failure is a missing import, it delegates a slave to locate the missing module. If the failure is an assertion mismatch, it delegates a slave to inspect the function under test. Each slave output informs the next delegation.

The advantage of reactive orchestration is its **adaptability**. The master does not commit to a fixed plan; it responds dynamically to the structure of the problem as it emerges. This reduces upfront planning overhead and avoids the pitfall of executing a plan that turns out to be inappropriate. The disadvantage is increased latency: each delegation requires a round-trip to a slave, and the master cannot parallelize tasks that might have been identified through upfront planning.

Reactive orchestration also places greater demands on the master's decision-making. Without a plan to guide it, the master must decide at each step whether it has enough information to proceed, whether it needs to investigate further, or whether it should terminate. These decisions require either sophisticated heuristics or a powerful decision model — often the master itself is a large language model capable of reasoning about partial information and uncertain outcomes.

Figure 17 illustrates the reactive orchestration pattern. A central master delegates tasks to slaves and receives three types of responses: success (the slave completed its task, and the master can continue or terminate), error (the slave failed, triggering delegation to a diagnostic agent), and unexpected (the slave discovered something that requires replanning). These three branches represent the core reactive logic: **adapt to what you learn, do not commit to what you assumed**.

16 Serial Agent Chains

In a serial agent chain, slaves execute sequentially, forming a pipeline topology. The output of Slave A becomes the input to Slave B, which feeds Slave C, and so on. The master coordinates handoffs between stages, ensuring each slave receives the correct input at the appropriate time.

This topology maps naturally onto transformation chains, where data must pass through multiple processing stages in a fixed order. For example, a document processing pipeline might parse raw text, analyze its structure, and format the output. Each stage depends on the previous stage's output, making parallelization impossible. Similarly, refinement chains implement sequential improvement: an initial draft is generated, reviewed for quality, and then revised based on feedback.

The serial topology offers predictability and simplicity. Dependencies are explicit and linear. Debugging is straightforward because you can inspect the data flowing between each stage. The primary limitation is latency: total execution time equals the sum of all stage durations, with no opportunity for parallel speedup.

The serial topology requires no dedicated figure; it is simply a chain of slaves where each stage's output becomes the next stage's input. The master coordinates handoff timing but does not participate in data transformation. Slaves operate independently once launched.

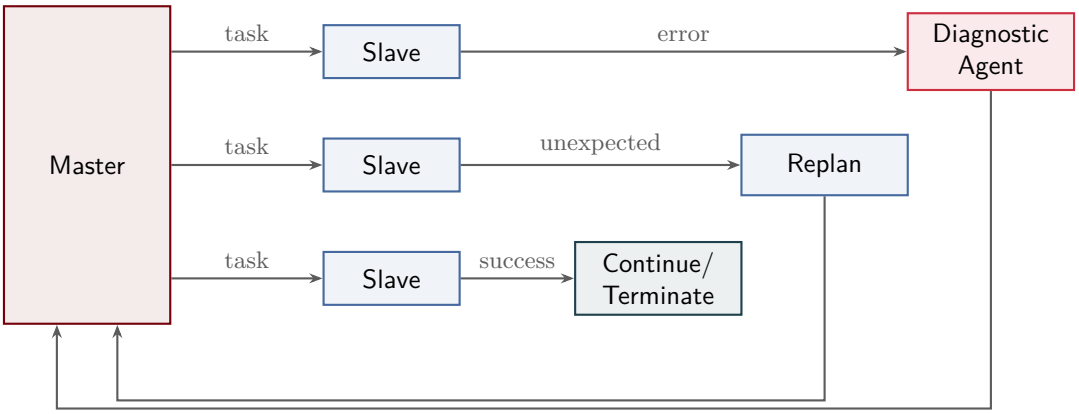

Figure 17: Reactive orchestration pattern. The master delegates tasks and reacts to three types of outcomes: unexpected (top row — slave discovers something surprising, triggering replanning and a new orchestration cycle), error (middle row — slave fails, diagnostic agent investigates and feeds back to the master), and success (bottom row — slave completes, master continues or terminates).

17 Parallel Agent Execution

Parallel agent execution enables slaves to operate simultaneously on independent tasks. The master fans out work and collects results. Unlike serial chains, parallel execution offers the potential for significant speedup when tasks are independent and slaves have sufficient computational resources.

The key challenge in parallel execution is synchronization: when should the master proceed with the results it has received? Three barrier types govern this decision, each suited to different requirements.

A **full barrier** waits for all slaves to complete before proceeding. This is required when every result is necessary for correctness. For example, when aggregating statistics across multiple data partitions, missing even one partition would corrupt the final result. Full barriers maximize correctness at the cost of latency: the slowest slave determines total execution time.

A **partial barrier** or k-of-n barrier proceeds when k slaves finish, where $k < n$. This pattern supports consensus mechanisms (majority vote requires $k > n/2$) and redundant task execution (take the first k results and discard the rest). Partial barriers reduce sensitivity to slow slaves but require careful design to ensure that proceeding with incomplete results does not violate correctness requirements.

A **timeout barrier** waits up to t seconds, then proceeds with whatever results are available. This implements best-effort execution: the system prioritizes responsiveness over completeness. Timeout barriers are appropriate when partial results have value and when strict deadlines matter more than exhaustive coverage. The master must handle missing results gracefully, either by synthesizing incomplete data or by flagging gaps in coverage.

Figures 18, 19, and 20 illustrate the three barrier types. Grayed slaves indicate those whose results are not required (or not yet available) for the master to proceed. The choice of barrier depends on the task's tolerance for incomplete results and its latency requirements.

18 Tree-Structured Agent Hierarchies

Tree-structured hierarchies extend the master-slave pattern with intermediate delegation layers. A root master delegates to lead agents, who in turn delegate to specialist slaves. This topology is about the shape of delegation, not about formal layering or rigid role definitions.

Information flows bidirectionally in a tree hierarchy. Tasks and context flow downward via solid edges: the root provides high-level goals to leads, who decompose them into subtasks for specialists. Results and status flow upward via dashed edges: specialists report to leads, who synthesize and report to the root. Each node acts as both a slave (to its master above) and a master (to its slaves below).

Tree hierarchies enable delegation at scale, but depth is limited by token costs. Each level multiplies total token consumption: if the root's prompt is

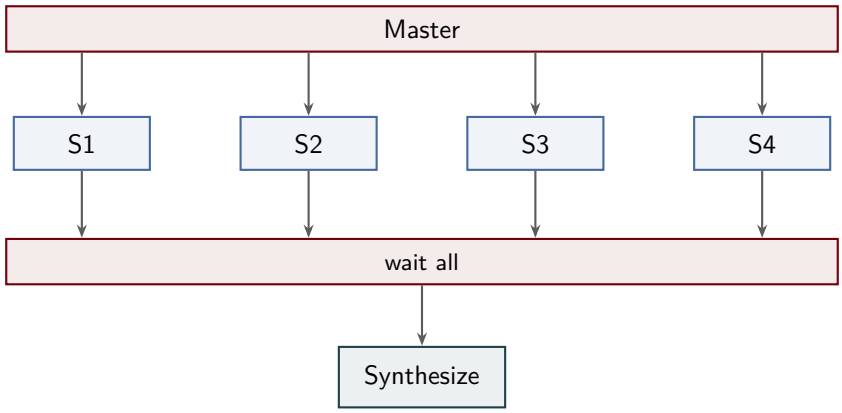

Figure 18: Full barrier: the master fans out tasks to all slaves and waits for every slave to complete before proceeding to synthesis. The slowest slave determines total execution time.

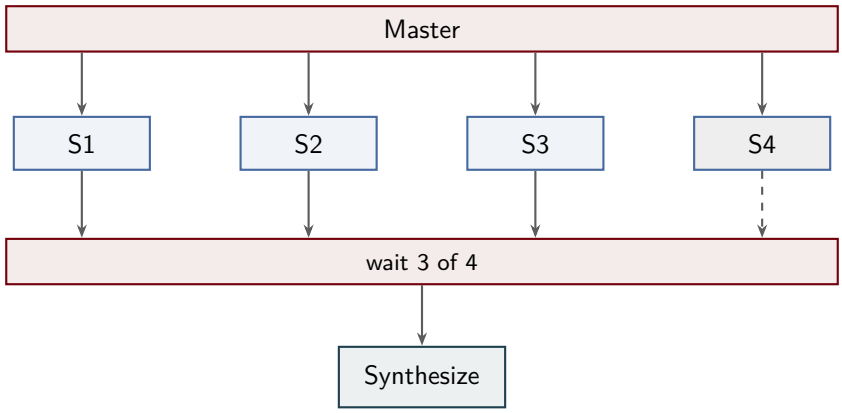

Figure 19: Partial barrier (k-of-n): the master proceeds when k slaves finish, here 3 of 4. The grayed slave (S4) may still be running but its result is not required (dashed arrow). This reduces sensitivity to slow slaves.

1000 tokens and each lead receives a similar context, and each specialist likewise, the total context across all agents can reach tens of thousands of tokens. Practical systems rarely exceed three levels due to these costs and the coordination overhead of deep hierarchies.

Figure 21 shows a three-level tree. The root delegates to three leads, each managing two specialists. Solid edges represent task delegation; dashed edges represent result aggregation. The root never communicates directly with specialist slaves; all communication is mediated by the leads.

19 Unidirectional vs. Bidirectional Communication

Communication between master and slave can be unidirectional or bidirectional. The choice affects protocol complexity, adaptability, and predictability.

In **unidirectional communication**, the master sends a task and the slave returns a result. No mid-task communication occurs. The slave cannot ask for clarification, request additional context, or negotiate scope. It must work with what it was given. This simplicity offers isolation and predictability: each slave operates independently, and the master's responsibilities are limited to task assignment and result collection.

Unidirectional communication is appropriate when tasks are well-specified and slaves have sufficient context to operate autonomously. The cost of this simplicity is rigidity: if the slave encounters ambiguity or missing information, it must make assumptions or fail. The master has no opportunity to intervene mid-task.

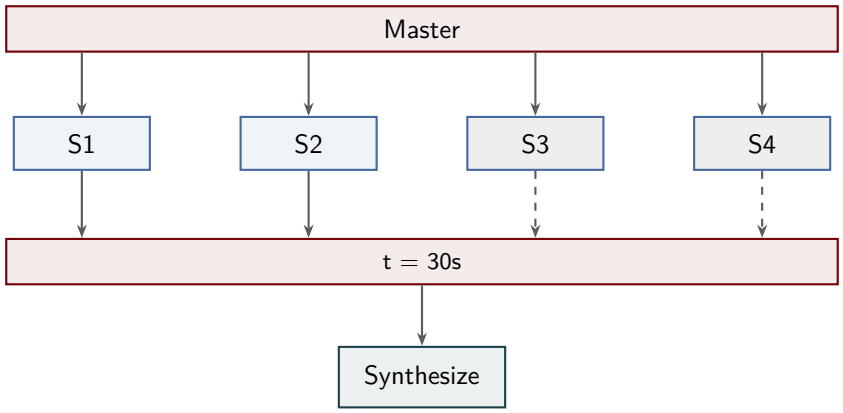

Figure 20: Timeout barrier: the master waits up to a fixed deadline (here 30 seconds), then proceeds with whatever results are available. Grayed slaves with dashed arrows have not yet completed. This prioritizes responsiveness over completeness.

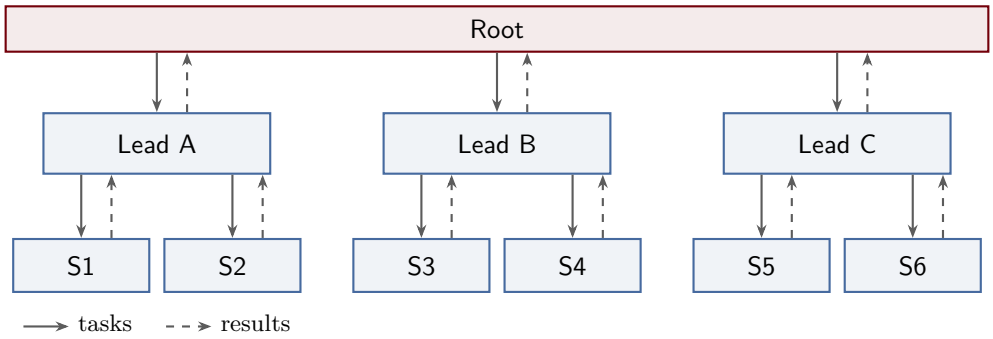

Figure 21: Tree-structured agent hierarchy with three levels. Tasks flow down (solid arrows), results flow up (dashed arrows). Each lead manages two specialists; the root never communicates directly with specialists.

In **bidirectional communication**, the slave can interrupt the master to ask questions, request clarification, or negotiate scope changes. This adaptability enables slaves to handle underspecified tasks and to refine their approach based on runtime discoveries. The master must handle interrupts, which complicates the orchestration protocol. Bidirectional communication increases coupling: the master and slave must agree on an interrupt protocol, and the master's availability becomes a bottleneck.

Figure 22 contrasts the two models. The unidirectional case shows a clean request-response cycle. The bidirectional case adds mid-task clarification (step 2) and additional context (step 3), shown in a distinct color to emphasize the interrupt protocol. Sequence numbers indicate the temporal ordering of messages.

20 Canonical Orchestration Patterns

The building blocks introduced in previous sections combine to form four canonical orchestration patterns. Each pattern represents a distinct solution to a recurring coordination problem.

The **pipeline** pattern combines serial topology, single-pass orchestration, and unidirectional communication. It implements transformation chains where data flows through sequential stages. Each stage performs a well-defined transformation and passes results forward. The pattern is simple and predictable but offers no parallelism.

The **fan-out/fan-in** pattern combines parallel topology, single-pass orchestration, and unidirectional communication. The master fans out independent subtasks to multiple slaves and synthesizes their results. This pattern exploits task parallelism for speedup but requires that subtasks be indepen-

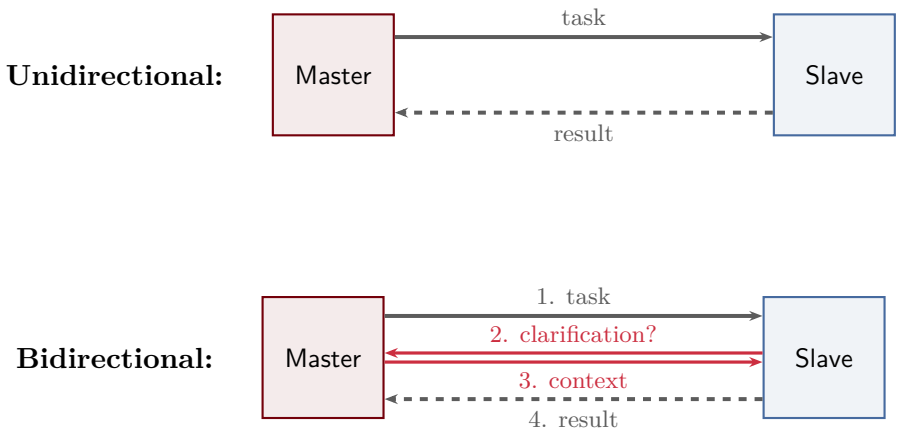

Figure 22: Unidirectional communication (top) vs. bidirectional communication (bottom). In unidirectional mode, the master sends a task and receives a result. In bidirectional mode, the slave can request clarification mid-task (steps 2–3 in rose).

dent.

The **master-slave pool** pattern combines parallel topology, iterative orchestration, and unidirectional communication. The master maintains a queue of tasks and assigns them to slaves as they become available. Slaves execute tasks independently and return results. The master continues until the queue is empty. This pattern efficiently handles batch processing and load balancing.

The **feedback loop** pattern combines serial topology, iterative orchestration, and bidirectional communication. The planner generates a plan, the executor attempts it, and the executor reports results back to the planner for refinement. This pattern enables iterative improvement and adaptive planning but incurs higher coordination overhead.

Figures 23, 24, 25, and 26 visualize the four canonical patterns. Each diagram uses consistent node sizes and color coding to emphasize structural differences. Table 2 maps each pattern onto its constituent building blocks and identifies its primary use case.

These patterns are not mutually exclusive. Real systems often combine multiple patterns within a single architecture, nesting fan-out within a pipeline stage or embedding a feedback loop inside a worker pool. The value of canonical patterns lies in their composability: once you understand the primitives, you can combine them to build arbitrarily complex orchestration logic.

21 Why Multi-Layered Architectures?

A flat master model—one master managing everything—creates a bottleneck. As task complexity grows, the master's context fills with operational details: which files were changed, what errors occurred, what each slave reported. The master loses sight of the big picture. Consider an analogy: a CEO who personally writes deployment scripts has no time left for strategy. The fundamental problem is one of cognitive load. A single agent cannot simultaneously hold strategic vision, tactical planning, and execution details within a finite context window.

Layering separates concerns. Strategic decisions occur at the top layer, where the agent reasons about goals, constraints, and success criteria. Tactical planning happens in the middle, where goals decompose into concrete task graphs with dependencies. Execution happens at the bottom, where specialized slaves perform individual operations and report results. Each layer filters and abstracts information before passing it upward or downward. This separation is what allows multi-agent systems to scale beyond the limitations of a single context window.

Figures 27 and 28 contrast these two approaches. The flat architecture shows a single master overwhelmed by direct connections to every slave. The layered architecture distributes coordination across multiple planning agents, each responsible for a subset of slaves. The visual clarity of the layered

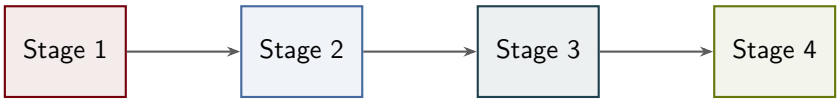

Figure 23: Pipeline pattern: sequential stages where each stage's output becomes the next stage's input. Serial topology, single-pass orchestration, unidirectional communication.

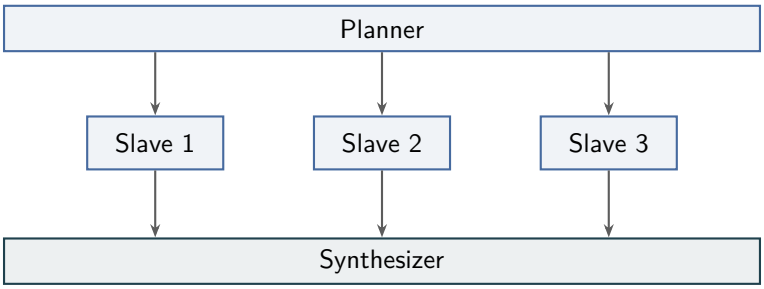

Figure 24: Fan-out/fan-in pattern: the planner distributes independent subtasks to slaves, then a synthesizer merges results. Parallel topology, single-pass orchestration, unidirectional communication.

diagram reflects the conceptual clarity it provides to the system.

22 Two-Layer Architecture

The simplest multi-agent structure consists of one master and multiple slaves. This is the architecture described in Sections 9–20. The master holds the big picture, decomposes tasks into subtasks, and synthesizes results. Slaves execute scoped tasks and report back. This division of labor is sufficient for many practical applications.

The limitations become apparent at scale. The master is still a single point of failure and a potential bottleneck. It must simultaneously maintain strategic context—what are we trying to achieve?—and tactical details—which slave is doing what, what has completed, what failed? For small-to-medium tasks with three to five subtasks, this dual responsibility is manageable. Beyond that, the master's context window becomes strained. It begins to lose track of the big picture as it drowns in execution details.

The left panel of Figure 29 illustrates the two-layer architecture. The master must allocate a substantial portion of its context window—approximately 80,000 tokens—to track both high-level goals and low-level slave state. Each slave operates with a smaller, task-specific context of roughly 30,000 tokens. This asymmetry is functional but places a heavy burden on the master.

23 Three-Layer Architecture

A three-layer architecture introduces an intermediate planning layer between strategy and execution. Each layer has a distinct responsibility and its own context budget:

- **Strategy layer:** *What are we trying to accomplish?* This layer reasons about high-level goals, constraints, and success criteria. It uses a small context budget—around 15,000 tokens—because it operates at a high level of abstraction. It never sees raw tool output or execution details.
- **Planning layer:** *How do we break this into tasks?* This layer receives strategic goals and decomposes them into a task graph with dependencies, sequencing constraints, and resource allocations. It uses a moderate context budget—approximately 50,000 tokens—sufficient to represent a complex task structure without drowning in execution minutiae.
- **Execution layer:** *Do the work.* Multiple agents at this layer execute individual tasks. Each receives a fresh context window containing only the information necessary for its specific task—typically 30,000 tokens. Slave agents never see the full strategic plan or the complete task graph. They operate on local information, which is precisely what allows them to scale.

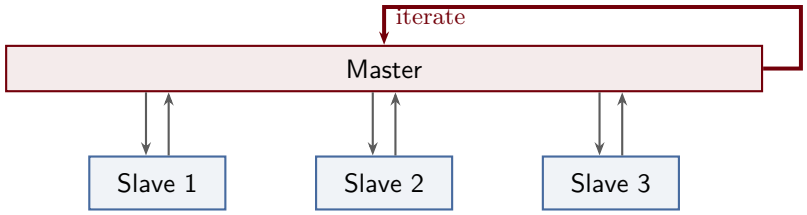

Figure 25: Master-slave pool pattern: the master assigns tasks from a queue, slaves execute and return results, and the cycle repeats. Parallel topology, iterative orchestration, unidirectional communication.

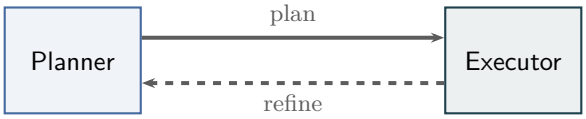

Figure 26: Feedback loop pattern: the planner generates a plan, the executor attempts it and reports results back for refinement. Serial topology, iterative orchestration, bidirectional communication.

The key insight enabling this architecture is **information filtering**. Each layer abstracts and summarizes before passing information up or down. The strategy layer sees task completion summaries, not raw logs. The execution layer sees task specifications, not strategic rationale. This filtering is not accidental or optional—it is the mechanism that allows multi-layered systems to transcend the context limitations of individual agents.

The right panel of Figure 29 shows the token budget allocation across three layers. Notice that the total token budget may be similar to or even less than the two-layer case, but the distribution is fundamentally different. No single agent must hold the entire system state. Each agent operates within a cognitively manageable scope.

24 When to Add Layers

How many layers should you use? The answer depends on task complexity, the number of subtasks, and the structure of dependencies. Guidelines:

- **1 layer (single agent):** Simple tasks with fewer than three subtasks, all of which fit comfortably within one context window. Example: analyzing a single file and generating a summary.
- **2 layers:** Most practical tasks. Three to five subtasks with moderate interdependencies. The master can track task state without becoming overwhelmed. Example: refactoring a module, running tests, and updating documentation.
- **3 layers:** Complex projects with ten or more subtasks, multiple dependency chains, and a clear distinction between strategic and tactical concerns. Example: designing, implementing, testing, and deploying a new feature across multiple services.
- **4+ layers:** Diminishing returns. Each additional layer adds coordination overhead, increases latency, and multiplies token costs. The practical ceiling for most real-world systems is three to four layers. Beyond that, the complexity of inter-layer communication outweighs the benefits of additional decomposition.

Figure 30 quantifies this tradeoff. The capability curve shows that most of the benefit comes from the first two layers. The third layer provides meaningful but diminishing gains. Beyond three layers, capability improvements are marginal. Meanwhile, coordination cost—measured in latency, token consumption, and debugging complexity—accelerates rapidly. The dashed line at three layers marks the empirical sweet spot for most applications.

Table 3 summarizes the scaling behavior. Notice that token consumption and latency grow faster than linearly with layer count. A four-layer system does not simply double the cost of a two-layer system—it triples or quadruples it. This superlinear growth is why depth is not the answer to every

Table 2: Canonical orchestration patterns and their building blocks.

Pattern	Topology	Orchestration	Direction	Best For
Pipeline	Serial	Single-pass	Unidirectional	Transformation chains
Fan-Out/Fan-In	Parallel	Single-pass	Unidirectional	Independent subtasks
Master-Slave Pool	Parallel	Iterative	Unidirectional	Batch processing
Feedback Loop	Serial	Iterative	Bidirectional	Iterative refinement

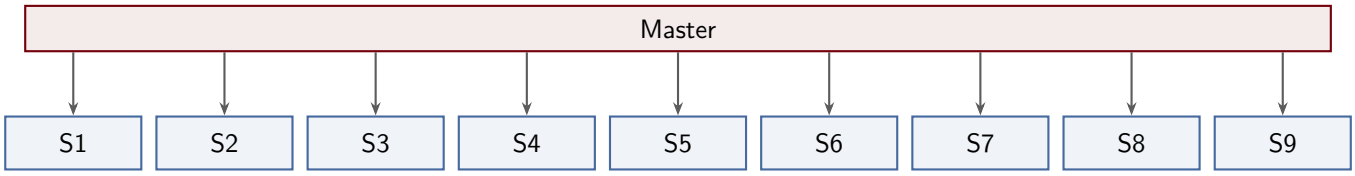

Figure 27: Flat architecture: a single master coordinates directly with every slave. As the number of slaves grows, the master's context fills with operational details, leading to context overload.

scaling problem.

25 Domain Specialization Across Layers

Instead of adding more layers—going *deeper*—you can add more specialists at the same level—going *wider*. A single master manages multiple domain experts: a code agent, a test agent, a documentation agent, a deployment agent. Each specialist has its own system prompt, tools, and context window. This horizontal scaling is often more effective than deep hierarchies because it reduces latency. Fewer handoffs mean faster execution. Specialization means each agent develops expertise within its domain rather than acting as a generalist bottleneck.

The tradeoff is that the master must understand enough about each domain to delegate effectively. If the master's context becomes overwhelmed by the breadth of domains—tracking what the code agent did, what the test agent found, what the documentation agent wrote, and what the deployment agent needs—that is the signal to add a planning layer above it. The master then delegates domain coordination to intermediate planning agents, each responsible for a subset of specialists.

Figure 31 contrasts deep and wide architectures. The deep system on the left requires four sequential handoffs to reach the execution layer. Each handoff adds latency and potential for information loss. The wide system on the right allows six specialists to operate in parallel, coordinated by a single master above them. The visual contrast is stark: depth is tall and narrow, width is short and broad. The choice between them depends on whether your bottleneck is decomposition complexity (which benefits from depth) or domain coverage (which benefits from width).

In practice, successful systems often combine both approaches. A three-layer architecture might have a strategic layer at the top, a planning layer that delegates to domain-specific masters in the middle, and domain specialists at the bottom. The strategic layer goes deep to separate concerns. The domain specialists go wide to cover diverse technical areas. This hybrid structure balances the competing demands of abstraction, specialization, and coordination.

26 The Pipeline Pattern

The pipeline pattern organizes computation as a strictly sequential chain of transformations. Each stage receives input from the previous stage, performs a specific transformation, and passes the result to the next stage. Stages execute one after another—there is no parallelism within the pipeline itself. The master's role is minimal: it assembles the chain, initiates the first stage, and monitors for failures.

This pattern offers several advantages. The sequential structure makes pipelines simple to reason about and debug—each intermediate output can be inspected in isolation. Pipelines naturally fit trans-

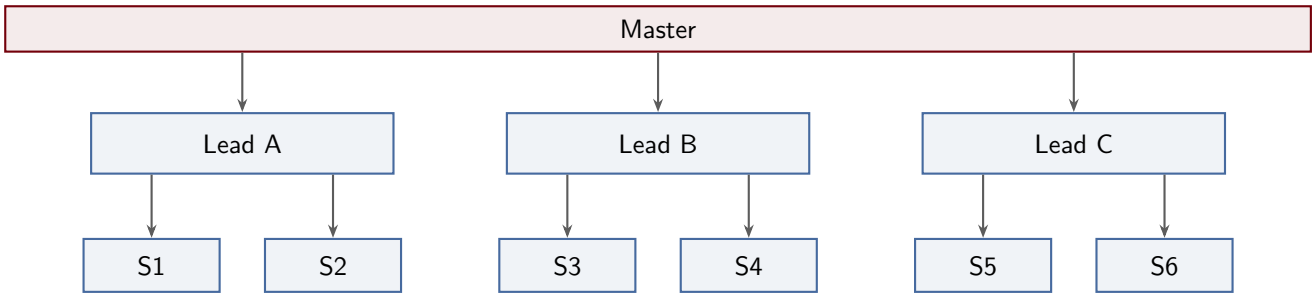

Figure 28: Layered architecture: the master delegates to intermediate leads, who each manage a subset of slaves. Each lead acts as both a slave (to the master above) and a master (to its slaves below). No arrows cross, and each agent has a manageable scope.

Figure 29: Comparison of two-layer and three-layer agent architectures. The two-layer system places all strategic and tactical reasoning in a single master, consuming a large context budget. The three-layer system separates strategy from planning, allowing each layer to operate within a smaller, more focused context window. The execution layer scales horizontally by adding slaves as needed.

formation workflows where each step builds on the previous result: data ingestion, parsing, analysis, formatting, and output. The clear separation of concerns allows each stage to be developed, tested, and maintained independently.

However, pipelines have inherent limitations. Total latency is the sum of all stage latencies—no overlap is possible. A failure at any stage blocks the entire pipeline, requiring careful error handling at each boundary. For tasks with independent subtasks, the pipeline's strict sequencing wastes potential parallelism. Despite these constraints, pipelines remain a fundamental building block for workflows that require ordered transformations.

Figure 32 (top) illustrates a five-stage pipeline for document processing. Raw input enters the Ingest stage, producing structured data for Parse, which feeds analyzed content to Analyze, then formatted output to Format, and finally the polished result to Output. The master establishes the chain and monitors each stage, but the data flows strictly left-to-right through the sequence.

27 The Fan-Out/Fan-In Pattern

The fan-out/fan-in pattern exploits task parallelism by decomposing work into independent subtasks, executing them concurrently, and merging the results. A planner examines the overall task, identifies subtasks that can proceed without coordination, and distributes them to a set of parallel slaves. Each slave executes its subtask independently—no inter-slave communication is required. A synthesizer collects all slave outputs and combines them into a unified result.

This pattern is ubiquitous in tasks with natural parallelism. Literature reviews fan out queries to multiple databases or search engines, then synthesize the combined bibliography. Code reviews fan out file-level analyses to separate slaves, then merge

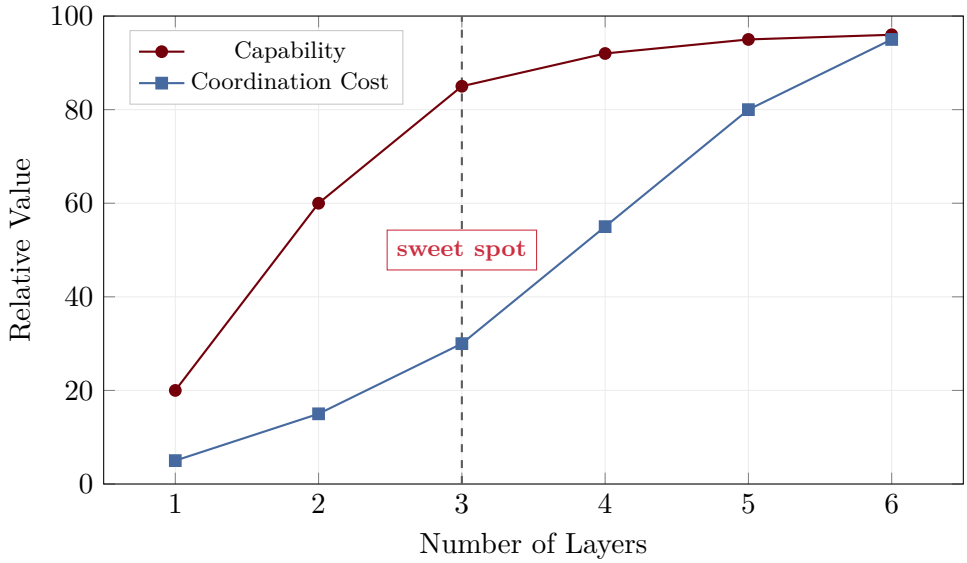

Figure 30: Cost-benefit analysis of adding layers to agent architectures. Capability increases rapidly from one to three layers, then plateaus. Coordination cost grows slowly at first but accelerates beyond three layers. The intersection represents the practical sweet spot for most systems.

Table 3: Scaling characteristics by layer count.

Layers	Typical Agents	Approx. Tokens	Relative Latency
1	1	50K	1×
2	4–6	150–200K	2×
3	8–15	300–500K	3–4×
4	20+	600K+	5–8×

findings. Testing frameworks fan out test suites to parallel executors, then aggregate pass/fail results. The key enabling assumption is subtask independence: each slave must be able to complete its work without information from other slaves.

The planner's decomposition step is critical. If subtasks have hidden dependencies—shared state, ordering constraints, or required coordination—results may be inconsistent or incomplete. Conversely, correct decomposition yields significant speedup: total latency becomes the maximum slave latency rather than the sum of all subtask latencies. The pattern also provides natural fault tolerance: if a slave fails, the master can retry that subtask without affecting other slaves.

Figure 32 (bottom) shows a fan-out/fan-in workflow. The planner distributes three subtasks to three slaves operating in parallel. Each slave independently completes its assignment and returns results to the synthesizer, which merges them into a final output. The master monitors both planning and synthesis but does not participate in subtask execution.

28 The Master-Slave Pool Pattern

The master-slave pool pattern manages large-scale batch processing through a centralized task queue and a pool of interchangeable slaves. A master agent maintains the queue, adding tasks as they arrive or as prior results generate new work. Slaves pull tasks from the queue, execute them, return results to the master, and immediately request the next task. Slaves are stateless and interchangeable—if one slave fails, another can retry the same task. The master tracks completion and decides when the queue is empty or a stopping condition is met.

This pattern excels at throughput-oriented workflows. Analyzing thousands of log files, running extensive test suites, processing batches of data

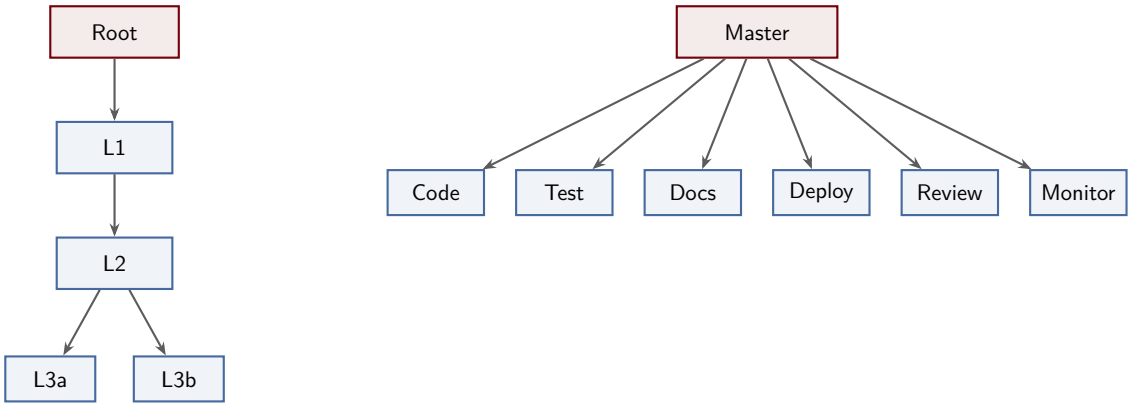

Figure 31: Deep versus wide agent architectures. Deep systems have many layers with narrow branching, resulting in high latency due to sequential handoffs. Wide systems have few layers with broad branching, allowing parallel execution across specialized domains. Each approach has its place: depth for complex decomposition, width for domain coverage.

records, or generating multiple variations of a design all fit naturally into the master-slave model. The pool size can scale dynamically: add slaves to increase throughput, remove slaves when load decreases. The master can implement sophisticated scheduling policies: prioritize high-value tasks, load-balance across slaves, or retry failed tasks with backoff.

The iterative nature distinguishes this pattern from simple fan-out. The master may add new tasks to the queue based on results from completed tasks, creating a feedback loop. For example, a web crawler's master enqueues initial seed URLs; as slaves fetch pages, they report discovered links, which the master adds to the queue. This dynamic task generation continues until no new work is discovered or a resource limit is reached.

Figure 33 (top) illustrates the master-slave pool. The master enqueues tasks `T1`, `T2`, `T3`, `T4`, and so on into a central queue. Four slaves dequeue tasks, execute them, and return results to the master. The master can dynamically add new tasks based on those results, creating an iterative workflow.

29 The Feedback Loop Pattern

The feedback loop pattern addresses tasks where initial attempts rarely achieve acceptable quality. Results from execution flow back to a planning agent for iterative refinement. The planner issues instructions, an executor produces a result, an evaluator judges quality, and failures trigger revised planning informed by specific deficiencies. This cycle continues until quality is sufficient or a maximum iteration count is reached.

Bidirectional communication between planner and executor distinguishes this pattern from simple retry logic. The evaluator does not merely return pass/fail—it provides detailed feedback about what went wrong, which aspects need improvement, and which constraints were violated. The planner uses this feedback to adjust its next set of instructions, focusing on the identified deficiencies rather than starting from scratch.

Common applications include code generation (write code, run tests, fix failures, repeat), document writing (draft text, review for clarity and accuracy, revise weak sections), and optimization (measure performance, identify bottlenecks, adjust parameters, re-measure). In each case, the first attempt provides valuable information that guides subsequent refinement.

The feedback loop's iterative nature has cost implications. Each iteration consumes tokens for planning, execution, and evaluation. Setting appropriate stopping conditions is critical: too few iterations may yield poor quality, while too many waste resources on diminishing returns. Effective evaluators provide actionable feedback that enables rapid convergence, reducing the total iteration count needed.

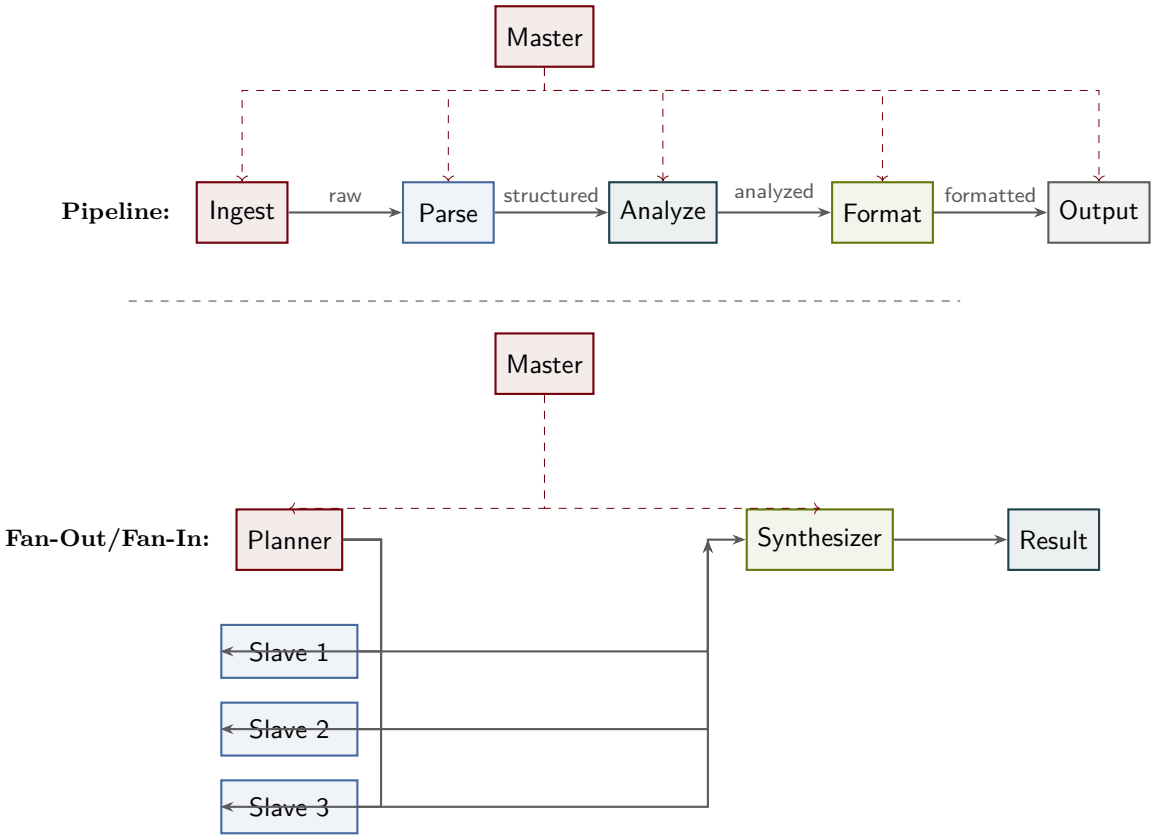

Figure 32: **Top:** The pipeline pattern processes data through a sequential chain of transformations. Each stage performs a specific operation and forwards the result to the next stage. The master coordinates the chain but does not participate in data transformation. **Bottom:** The fan-out/fan-in pattern distributes independent subtasks to parallel slaves. The planner decomposes the problem, slaves execute in parallel, and the synthesizer merges results.

Figure 33 (bottom) shows the feedback loop structure. The planner sends instructions to the executor, which produces a result for the evaluator. If the evaluator judges the result acceptable, the workflow terminates. Otherwise, the evaluator returns detailed feedback to the planner, which incorporates those deficiencies into revised instructions for the next iteration.

30 Combining Patterns

Real systems rarely use a single pattern in isolation. Complex workflows compose multiple patterns, each handling a phase suited to its strengths. A research pipeline might use fan-out for literature search, a sequential pipeline for data cleaning, a master-slave pool for running experiments, and feedback loops for manuscript writing. The master at the top selects and coordinates the appropriate pattern for each phase based on the task structure.

The key design principle is matching the pattern to the task's inherent characteristics. Tasks with independent subtasks benefit from fan-out parallelism. Sequential transformations where each step depends on the previous result require a pipeline. Iterative refinement toward a quality target needs a feedback loop. Large-scale batch processing with uniform work items calls for a master-worker pool. By analyzing task dependencies, ordering constraints, quality requirements, and scale, designers can select the pattern that best exploits the task structure.

Composition requires careful boundary management. Outputs from one pattern become inputs to the next, requiring compatible data formats and error-handling conventions. The top-level master manages these transitions, ensuring that failures in one pattern do not silently propagate to dependent patterns. State management becomes critical:

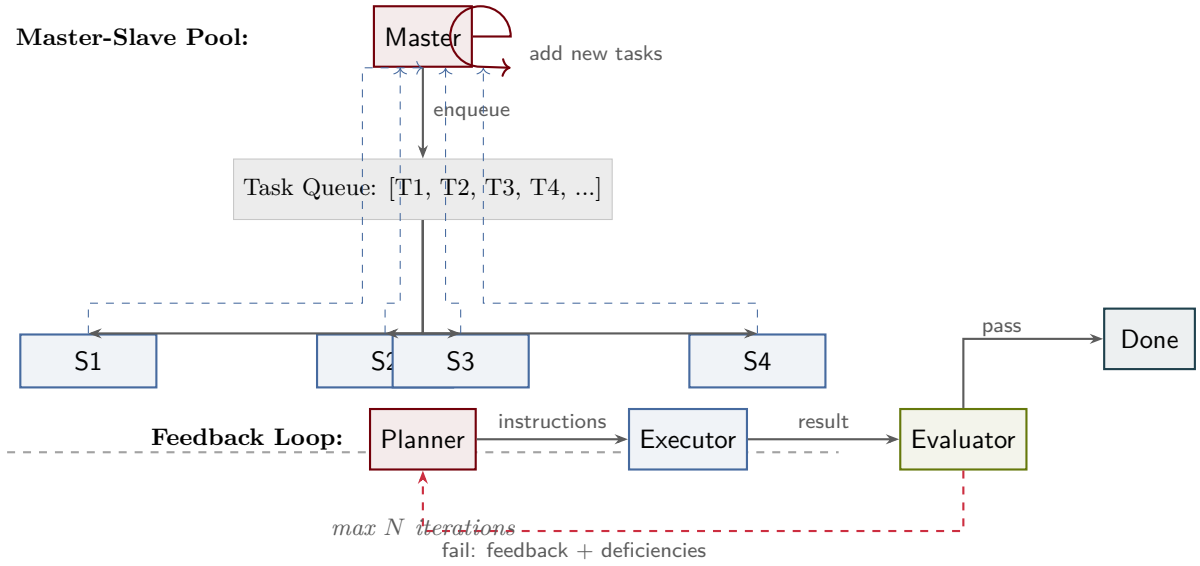

Figure 33: **Top:** The master-slave pool pattern uses a centralized task queue and stateless slaves. The master enqueues tasks, slaves dequeue and execute them, and results flow back to the master. The master can add new tasks based on results, enabling iterative workflows. **Bottom:** The feedback loop pattern refines results through iterative execution. The planner issues instructions, the executor produces a result, the evaluator judges quality, and failures trigger revised planning with feedback about deficiencies.

which pattern owns intermediate results, how are they cached or persisted, and who is responsible for cleanup.

Hybrid workflows also enable adaptive execution. The master can monitor pattern performance and switch strategies dynamically. A fan-out phase that discovers subtask dependencies might transition to a pipeline. A pipeline stage that encounters a large batch might spawn a master-slave pool for that stage alone. This flexibility allows systems to respond to workload characteristics discovered at runtime.

Figure 34 illustrates a composite research workflow. The top-level research master coordinates four phases, each using a different pattern. The literature phase fans out queries to multiple sources and merges results. The data phase pipelines ingestion, cleaning, and validation. The experiment phase uses a master-slave pool to run many trials in parallel. The writing phase employs a feedback loop to iteratively draft and revise the manuscript.

31 Pattern Selection Guide

Choosing the right pattern—or combination of patterns—requires analyzing the task structure along several dimensions. Are subtasks independent, or does each depend on results from others? Must steps execute in a specific order, or can they run concurrently? Is iteration needed to achieve acceptable quality? Is the workload large and uniform, or small and varied?

Independent subtasks with no ordering constraints favor fan-out/fan-in parallelism. If subtasks must execute sequentially—each transforming the output of the previous stage—a pipeline is appropriate. Tasks requiring iterative refinement call for a feedback loop, where each iteration incorporates lessons from prior attempts. Large-scale batch processing with uniform work items benefits from a master-slave pool, which maximizes throughput through dynamic load balancing.

Table 4 summarizes these considerations across seven dimensions. Task independence indicates whether subtasks can execute without coordination. Iteration captures whether quality improvement requires multiple passes. Scale reflects the typical number of subtasks: pipelines handle a few stages, fan-out manages moderate parallelism, and master-slave pools scale to hundreds of tasks. Quality emphasis distinguishes patterns optimized for throughput (master-slave) from those focused on

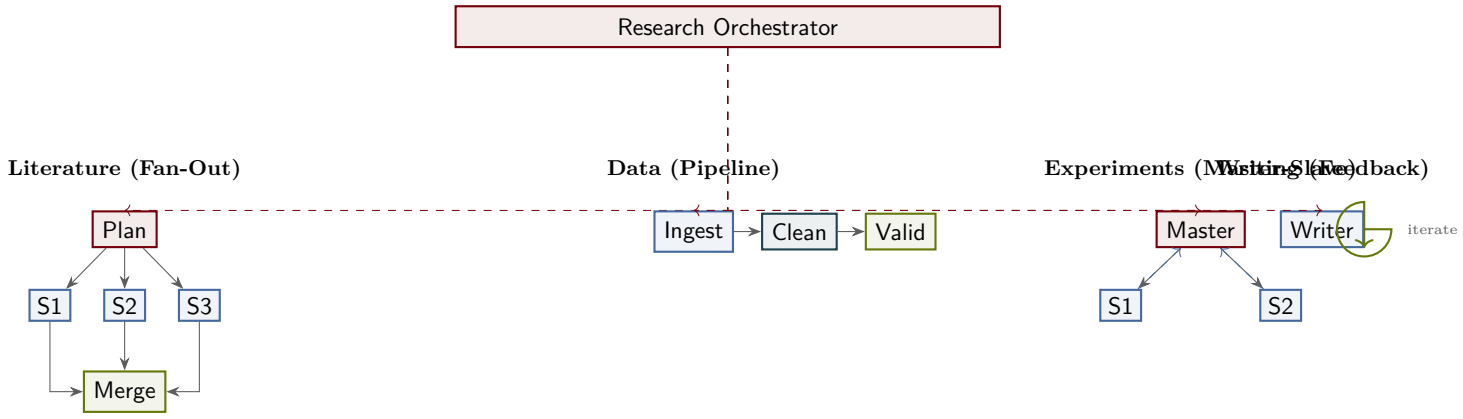

Figure 34: A composite research workflow combines four patterns under a single master. The literature phase uses fan-out to query multiple sources in parallel. The data phase pipelines ingestion, cleaning, and validation steps. The experiment phase employs a master-slave pool for high-throughput execution. The writing phase uses a feedback loop to iteratively refine the manuscript. Each pattern handles the phase best suited to its strengths.

refinement (feedback loop).

Token cost varies by pattern. Pipelines incur linear cost proportional to the number of stages. Fan-out patterns pay for subtasks in parallel but avoid duplication. Master-worker pools consume tokens proportional to the total workload. Feedback loops multiply costs by the number of iterations, making early convergence critical.

Complexity reflects both implementation difficulty and cognitive load during debugging. Pipelines are simplest to implement and reason about. Fan-out and master-slave patterns add coordination overhead. Feedback loops require careful evaluator design and stopping logic.

Latency characteristics also differ. Pipeline latency is the sum of all stage latencies. Fan-out latency is determined by the slowest worker. Master-slave pools batch tasks, trading some latency for throughput. Feedback loops multiply latency by iteration count.

The final row, "Best for," summarizes canonical use cases. Pipelines excel at sequential transformations. Fan-out handles independent parallel work. Master-slave pools optimize batch processing. Feedback loops enable iterative refinement toward quality targets.

By evaluating a task against these dimensions, designers can systematically select the pattern that best matches the task structure.

Figure 35 places these patterns within a broader taxonomy of agent architectures. At the top level, architectures divide into single-agent and multi-agent categories. Single-agent systems may operate in streaming, batch, or parallel modes, but they lack the coordination mechanisms that define multi-agent patterns.

Multi-agent systems further divide into flat and hierarchical organizations. Flat architectures distribute control: no agent has authority over others. The pipeline and fan-out/fan-in patterns are flat—the master coordinates but does not command. Hierarchical architectures establish explicit control relationships. Master-slave pools and feedback loops are hierarchical—the master or planner directs subordinate agents.

This taxonomy provides a roadmap for navigating design space. Starting from the task characteristics, designers first decide single-agent versus multi-agent, then flat versus hierarchical, and finally select the specific pattern that best fits the task structure.

32 Conclusion

We have traced a deliberate path from the simplest computational unit—an LLM as a text-in, text-out function—to increasingly sophisticated multi-agent systems. This journey did not require changing the LLM itself. Instead, we progressively wrapped the core model in layers of control: first with agents that observe and act, then with execution models that orchestrated parallel and streaming invoca-

Table 4: Pattern selection guide based on task characteristics.

Criterion	**Pipeline**	**Fan-Out/Fan-In**	**Master-Slave**	**Feedback Loop**
Task independence	Low	High	High	Low
Iteration needed	No	No	Optional	Yes
Scale (subtasks)	3–5	3–20	10–100+	2–5
Quality emphasis	Moderate	Moderate	Throughput	High
Token cost	Linear	Parallel	High	Moderate
Complexity	Low	Medium	Medium	Medium
Latency	Sum of stages	Max of workers	Batched	Iterations × stage
Best for	Transforms	Independent work	Batch processing	Refinement

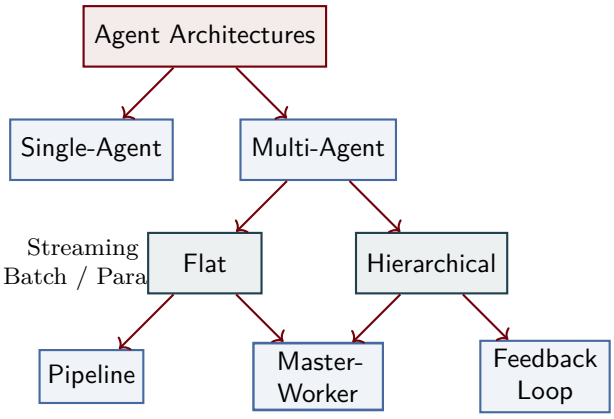

Figure 35: A taxonomy of agent architectures. Single-agent systems may operate in streaming, batch, or parallel modes but lack multi-agent coordination. Multi-agent systems divide into flat (pipeline, fan-out/fan-in) and hierarchical (master-worker, feedback loop) organizations. This taxonomy provides a structured approach to pattern selection based on control flow and task structure.

tions, next with slave hierarchies that decomposed complex problems, and finally with orchestration patterns that coordinated multiple independent systems. At each step, the model remained constant while the architecture around it grew in complexity and capability.

This insight is fundamental: the LLM is an uncommitted tool. Its power comes not from its individual inference but from how we compose multiple inferences within a structured process. The four canonical patterns we examined—pipeline, fan-out/fan-in, master-slave, and feedback loop—are the vocabulary of this composition. These patterns are not LLM-specific inventions; they echo classical concurrent and distributed computing. Their importance here is that they are small enough to understand completely, composable enough to combine into larger systems, and general enough to apply across an enormous range of agent tasks. A system that orchestrates document analysis, code review, or scientific hypothesis testing may use these same patterns, differing only in the details of task decomposition and resource constraints.

Selecting the right architecture is not primarily a question of model capability. Given a task, the architecture emerges from the task's structure: its inherent parallelism, its feedback requirements, its hierarchical depth, and the latency constraints we impose. A well-designed agent system for real-time reasoning may use tight feedback loops and shallow hierarchies. A batch system for asynchronous analysis may embrace deep pipelines and wide fan-outs. Neither choice is superior; each is the natural fit to its problem. This decoupling between model selection and architectural design frees practitioners to reason about systems clearly.

As LLM-based applications move from research prototypes to production systems at scale, these patterns will become foundational. We can already see them emerging in production orchestration frame-

works and multi-agent platforms. Understanding them deeply—not merely as abstract diagrams but as concrete execution models with resource, latency, and consistency implications—is essential for anyone building intelligent systems. The progression we have outlined, from simple agents to complex hierarchies, provides both a map and a toolkit for that future. The LLM remains at the heart of the system, but the architecture around it is where the real engineering begins.